

1주 · 리눅스 & Docker

리눅스 커맨드 · 셸 스크립트 · Docker · Compose (5차시, 20H)

1-3 · 이미지, 컨테이너에 대한 이해와 도커 사용하기

1-3 · 이미지, 컨테이너에 대한 이해와 도커 사용하기

이미지와 컨테이너의 관계를 이해하고, 도커 기본 명령으로 nginx와 PostgreSQL 컨테이너를 직접 다룹니다.

학습 목표

- 가상머신과 컨테이너의 차이, 이미지와 컨테이너의 관계를 설명할 수 있습니다.
- `docker run`·`ps`·`logs`·`exec`·`stop`·`rm`으로 컨테이너 생명주기를 다룰 수 있습니다.
- `-p` 포트 매핑과 `-e` 환경변수, `-v` 볼륨 옵션의 역할을 이해합니다.
- PostgreSQL 컨테이너를 볼륨과 함께 띄우고 `psql`로 접속할 수 있습니다.

시간 배분 (4H)

구분	Chapter	시간
이론	이미지와 컨테이너 개념	1H
실습	도커 기본 명령 실습	1.5H
실습	볼륨과 포트로 데이터 다루기	1H
정리·회고	관계도 정리·컨테이너 정돈·질의응답	0.5H

참고 사전 준비: **Windows**는 Docker Desktop(설치 때 WSL2 연동을 켭니다) 또는 WSL2 우분투 안의 Docker Engine, **macOS**는 Docker Desktop, **리눅스(우분투)**는 `sudo apt`로 설치한 Docker Engine이 실행 중이어야 합니다.

세 경우 모두 터미널에서 `docker version` 을 실행해 Client와 Server 정보가 나오는지 미리 확인해 주세요.

설치는 [환경 설정] 페이지에 OS별로 나뉘어 있습니다.

오늘 만들 것

컨테이너는 "제 컴퓨터에서는 되는데요"를 끝내는 기술입니다.

프로그램만 보내는 것이 아니라, 그 프로그램이 도는 데 필요한 환경까지 한 상자에 담아 보내기 때문에 어느 컴퓨터에서 열어도 똑같이 돌아갑니다.

이번 과정에서 앞으로 띄울 모든 것 — MinIO · Spark · Kafka · Qdrant — 이 전부 컨테이너라서, 오늘이 나머지 7주의 입구입니다.

오늘은 남이 만든 이미지를 가져다 쓰는 데까지 합니다.

내 프로그램을 이미지로 만드는 것은 내일(1-4), 여러 컨테이너를 한꺼번에 다루는 것은 모레(1-5)입니다.

실습 전에 볼 것

「도커 기본 명령 실습」 페이지 맨 앞에 짧은 이론 네 가지가 있습니다.
명령을 치기 전에 읽고 시작하면 오류 메시지가 읽힙니다.

- **도커는 두 조각이다** — 내가 치는 docker 와 실제로 일하는 데몬. 「Cannot connect to the Docker daemon」 은 명령이 틀린 게 아니라 일할 쪽이 꺼진 것
- **명령 이름이 두 가지로 보이는 이유** — docker ps 와 docker container ls 는 같은 명령
- **컨테이너에는 상태가 있다** — 멈춘 것(exited)은 사라진 것이 아니라서 이름을 붙들고 있다
- **이미지 이름은 저장소:태그** — latest 는 「최신」 이 아니라 그냥 기본 태그 이름

이미지와 컨테이너 개념 (1H)

이미지 · 컨테이너 · 레지스트리 · 볼륨 · 포트 — 앞으로 매일 쓸 다섯 낱말의 뜻을 정확히 잡습니다.

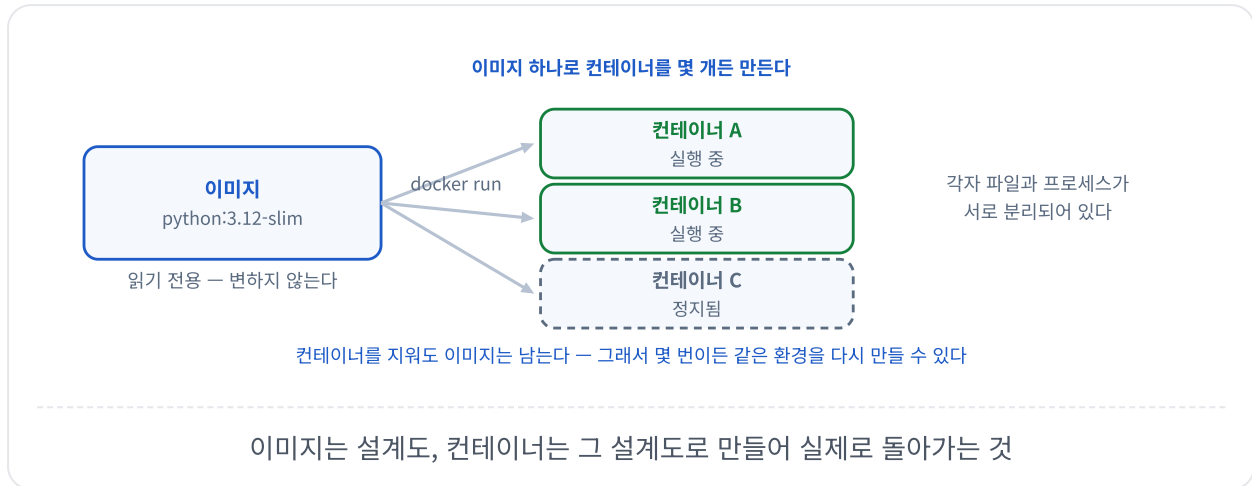
컨테이너가 푸는 문제

같은 파이썬 코드가 내 노트북에서는 돌고 서버에서는 안 도는 일이 흔합니다.

파이썬 버전이 다르거나, 설치된 라이브러리 버전이 다르거나, OS에 어떤 시스템 패키지가 없기 때문입니다.

컨테이너는 코드뿐 아니라 그 코드가 필요로 하는 것 전부를 함께 포장해 이 문제를 없앱니다.

이미지와 컨테이너 — 붕어빵 틀과 붕어빵



- 이미지(Image) = 붕어빵 틀입니다. 읽기 전용이고 변하지 않습니다. 파일 시스템 한 벌이 통째로 담겨 있습니다.
- 컨테이너(Container) = 그 틀로 찍어낸 붕어빵입니다. 실제로 실행되는 프로세스이고, 각자 독립된 파일·네트워크를 가집니다.
- 틀 하나로 붕어빵을 몇 개든 찍을 수 있듯, 이미지 하나로 컨테이너를 몇 개든 만들 수 있습니다.
- 붕어빵을 먹어 없애도(컨테이너 삭제) 틀은 남습니다. 그래서 언제든지 같은 환경을 다시 만들 수 있습니다.

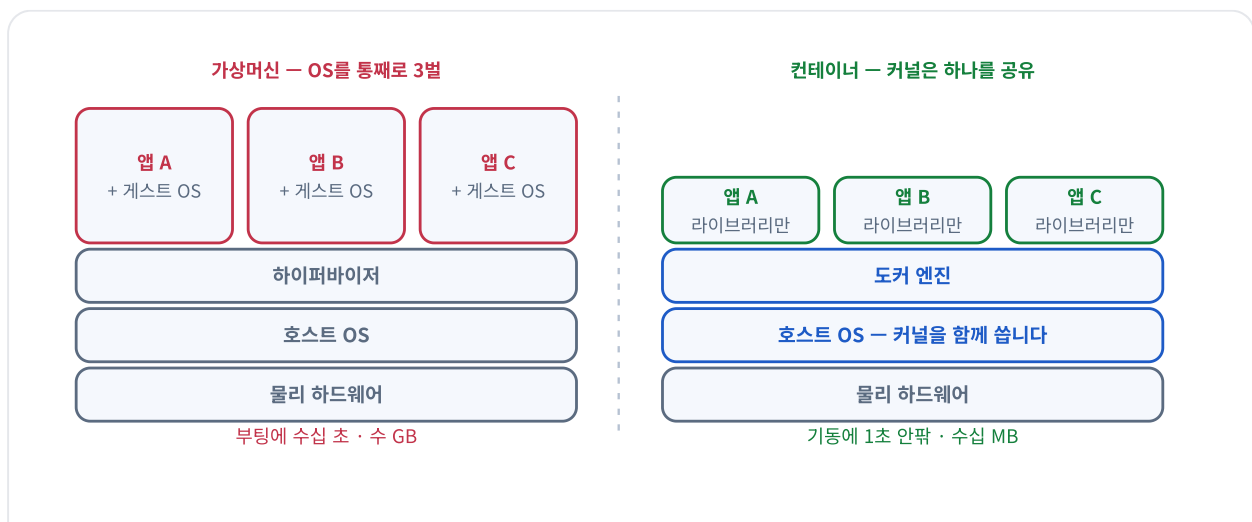
TIP 이 구분이 헛갈리면 명령어로 기억하세요.

docker images 는 틀 목록, docker ps 는 지금 구워지고 있는 붕어빵 목록입니다.

가상머신과는 무엇이 다른가

컨테이너를 "가벼운 가상머신"이라고 부르는 사람이 많은데, 정확히는 다른 물건입니다.

가장 큰 차이는 운영체제를 새로 올리느냐입니다.



가상머신은 건물을 통째로 새로 짓고, 컨테이너는 같은 건물에 칸막이 방을 만든다

가상머신		컨테이너
격리 방식	하드웨어를 흉내 내 OS를 통째로 올림	호스트 커널을 공유하고 프로세스만 격리
크기	수 GB	수십 MB ~ 수백 MB
기동 시간	수십 초 (부팅)	1초 안팎
한 대에 몇 개	보통 수 개	수십~수백 개
언제 쓰나	다른 OS가 필요할 때 (리눅스 위 윈도우 등)	같은 커널에서 앱만 격리하면 될 때

주의 커널을 공유한다는 말에는 제약이 따릅니다.

리눅스 컨테이너는 리눅스 커널 위에서만 돛니다.

맥이나 윈도우에서 도커가 도는 것은 Docker Desktop이 안쪽에 작은 리눅스 VM을 하나 띄워 주기 때문입니다.

그래서 맥·윈도우에서는 컨테이너 성능이 리눅스 서버보다 조금 낮습니다.

레지스트리 — 이미지를 받아 오는 창고

- Docker Hub(hub.docker.com)가 가장 큰 공개 레지스트리입니다. docker pull 은 여기서 틀을 내려받는 일입니다.
- 이미지 이름은 저장소:태그 형태입니다. postgres:16 에서 postgres 가 저장소, 16 이 태그(버전)입니다.
- 태그를 생략하면 latest 가 붙습니다. 실무에서는 latest 를 쓰지 않습니다 — 어제와 오늘의 latest 가 다른 이미지일 수 있어 재현성이 깨집니다.
- 회사에서는 사내 레지스트리(AWS ECR, Harbor 등)를 두고 자기 이미지를 올려 씁니다.

주의 이번 과정의 실습은 전부 태그를 명시합니다(postgres:16 · apache/kafka:3.7.0 처럼).

강의안에 적힌 태그를 그대로 쓰세요.

태그를 바꾸면 명령 옵션이 달라져 실습이 어긋날 수 있습니다.

포트 매핑 — 밖에서 안으로 들어가는 문

컨테이너는 자기만의 네트워크를 가집니다.

안에서 웹서버가 80번 포트로 떠 있어도, 밖(내 브라우저)에서는 그냥 보이지 않습니다.

-p 옵션이 그 사이에 통로를 뚫습니다.

bash

```
1 # -p 내컴퓨터포트:컨테이너포트
2 docker run -p 8080:80 nginx
3 #      ^^^^ ^^
4 #      |    +--- 컨테이너 안에서 nginx 가 듣고 있는 포트
5 #      +----- 내 브라우저가 접속할 포트 (localhost:8080)
```

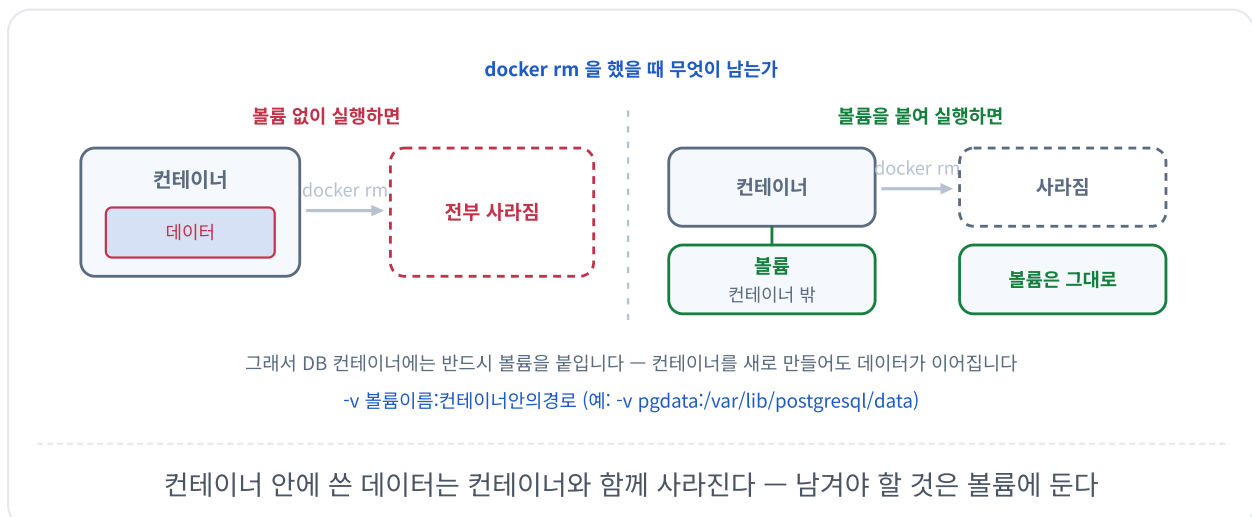
- 왼쪽(내 쪽) 번호는 내 마음대로 바뀌도 됩니다. 이미 쓰고 있는 번호만 피하면 됩니다.
- 오른쪽(컨테이너 쪽) 번호는 그 이미지가 정해 둔 것이라 바꿀 수 없습니다. Docker Hub 문서에 적혀 있습니다.
- -p 를 아예 안 쓰면 컨테이너는 돌지만 밖에서 접속할 수 없습니다.

볼륨 — 남겨야 할 것을 컨테이너 밖에 둡니다

컨테이너 안에 쓴 파일은 컨테이너를 지우면 함께 사라집니다.

이게 기본 동작이고, 대부분의 경우에는 오히려 좋은 성질입니다.

하지만 데이터베이스처럼 남아야 하는 것에는 곤란합니다.



- 이름 볼륨: `-v pgdata:/var/lib/postgresql/data` — 도커가 관리하는 저장 공간에 이름을 붙여 씁니다. 실습에서 주로 쓰는 방식입니다.
- 바인드 마운트: `-v ~/data-eng/week1:/work` — 내 컴퓨터의 폴더를 컨테이너 안에 그대로 연결합니다. 내 코드를 컨테이너에 넣어 돌릴 때 씁니다.
- 둘 다 `-v` 로 쓰지만, 앞이 이름이면 볼륨, 경로(`/` 로 시작)면 바인드 마운트입니다.

이 1시간에서 꼭 가져갈 것

- 이미지는 틀(읽기 전용), 컨테이너는 그 틀로 만든 실행 중인 것.
- 컨테이너는 커널을 공유하므로 가상머신보다 훨씬 가볍고 빠르다.
- 이미지는 저장소:태그로 부르고, `latest` 는 쓰지 않는다.
- `-p` 로 밖에서 안으로 들어갈 문을 뚫는다.
- 남아야 할 데이터는 `-v` 로 컨테이너 밖에 둔다.

도커 기본 명령 실습 (1.5H)

컨테이너를 띄우고, 안을 들여다보고, 정지하고, 지우는 한 바퀴를 손에 익힙니다.

명령을 치기 전에 — 도커는 두 조각입니다

앞 시간에 이미지와 컨테이너가 무엇인지 봤습니다.

이제 명령을 치는데, **내가 치는 docker 는 실제로 일하는 쪽이 아닙니다.**

도커는 **명령을 받는 쪽(클라이언트)**과 **실제로 일하는 쪽(데몬)**으로 나뉩니다.

내가 `docker run` 을 치면 그것은 **요청**일 뿐이고, 이미지를 받아 오고 컨테이너를 띄우는 일은 뒤에서 도는 데몬이 합니다.

식당에서 주문을 받는 사람과 요리하는 사람이 따로 있는 것과 같습니다.

조각	무엇인가	내 컴퓨터에서
클라이언트	내가 치는 <code>docker</code> 명령. 요청을 보낼 뿐이다	터미널
데몬 (dockerd)	이미지를 받고 컨테이너를 띄우는 본체	Docker Desktop / 리눅스 서비스

주의 이것을 알아야 오류를 읽을 수 있습니다.

「Cannot connect to the Docker daemon」 은 **명령이 틀린 것이 아니라 일할 쪽이 꺼져 있다**는 뜻입니다.

바로 아래 1단계에서 `docker version` 을 치는 이유가 이것입니다 — Client 는 나오는데 Server 가 안 나오면 데몬이 안 떠 있는 것입니다.

Windows·macOS 는 Docker Desktop 을 켜고, 리눅스는 `sudo systemctl start docker` 로 켵니다.

명령 이름이 두 가지로 보이는 이유

인터넷에서 찾아보면 같은 일을 하는데 **이름이 다른 명령**이 나옵니다.

도커가 중간에 명령 체계를 「`docker` 대상 동작」 꼴로 정리했는데, **옛 짧은 이름도 그대로 두었기** 때문입니다.

둘은 같은 명령입니다.

이 과정에서는 짧은 쪽을 씁니다.

짧은 옛 이름 (이 과정에서 씀)	정리된 이름	하는 일
docker ps	docker container ls	도는 컨테이너 목록
docker ps -a	docker container ls -a	멈춘 것까지 전부
docker images	docker image ls	받아 둔 이미지 목록
docker rm	docker container rm	컨테이너 지우기
docker rmi	docker image rm	이미지 지우기

컨테이너에는 상태가 있습니다

컨테이너는 **있거나 없거나** 둘 중 하나가 아닙니다.

상태가 있고, 명령이 그 상태를 옮깁니다.

이걸 모르면 **「지웠는데 목록에 또 있다」** 나 **「이름이 이미 쓰이고 있다」** 는 오류를 만났을 때 손을 못 땁니다.

상태	뜻	여기서 나가려면
created	만들어졌지만 아직 안 돌아다	<code>docker start</code>
running	돌고 있다. <code>docker ps</code> 에 보인다	<code>docker stop</code>
exited	멈췄다. 사라진 것이 아니다 — <code>docker ps -a</code> 에 보인다	<code>docker start</code> 로 되살리거나 <code>docker rm</code> 으로 지운다
(없음)	<code>docker rm</code> 으로 지워졌다	다시 <code>docker run</code> 하면 새로 만들어진다

TIP 가장 자주 만나는 오류가 「The container name ... is already in use」 입니다. 같은 이름의 컨테이너가 exited 로 남아 있다는 뜻입니다 — 지워진 것이 아니라 멈춰 있는 것이어서 이름을 아직 붙들고 있습니다.

`docker ps -a` 로 확인하고 `docker rm` 이름으로 치운 뒤 다시 만들면 됩니다.

도는 중이면 `docker rm -f` 이름으로 한 번에 멈추고 지울 수 있습니다.

이미지 이름은 저장소:태그 입니다

아래 2단계에서 `nginx:1.27` 을 받습니다.

콜론 앞이 저장소 이름, 뒤가 태그(판) 입니다.

태그를 안 적으면 `latest` 가 붙습니다.

적은 것	실제로 받는 것
nginx	nginx:latest
nginx:1.27	1.27 판
postgres:16	16 판

주의 `latest` 는 「가장 최신」 이라는 뜻이 아니라 **그냥 기본 태그 이름**입니다.

누가 어떤 판에든 붙일 수 있고, 어제 받은 latest 와 오늘 받은 latest 가 다를 수 있습니다.

그래서 실습이든 실무든 **판 번호를 적어** 씁니다 — 이 과정의 모든 예제가 nginx:1.27 ·

postgres:16 처럼 번호를 달고 있는 이유입니다.

`docker pull` 을 하면 여러 줄이 동시에 내려오는 것이 보입니다.

이미지가 **여러 겹(레이어)으로 나뉘어** 있어서 겹마다 따로 받기 때문입니다.

이미 받아 둔 겹은 다시 받지 않아 「Already exists」 로 지나갑니다 — 두 번째 pull 이 빠른 이유입니다.

겹이 어떻게 쌓이는지는 내일(1-4) Dockerfile 에서 다룹니다.

1단계 — 도커가 제대로 도는지 확인합니다

bash

```
1 docker version
2 docker run hello-world
```

확인 포인트: docker version에 Client·Server 정보가 모두 나오고, hello-world 실행 시 "Hello from Docker!" 메시지가 출력되면 성공입니다.

Server 정보가 안 나오면 도커 데몬이 안 떠 있는 것입니다.

Docker Desktop을 실행하고 아이콘이 켜질 때까지 기다린 뒤 다시 시도하세요.

2단계 — 이미지를 받아 보고 목록을 확인합니다

bash

```
1 # 이미지 내려받기 (실행은 하지 않습니다)
2 docker pull nginx:1.27
3
4 # 내 컴퓨터에 있는 이미지 목록
5 docker images
6
7 # 이미지 크기를 보기 좋게
8 docker images nginx
```

확인 포인트: docker images 목록에 nginx 가 태그 1.27 로 보이고 SIZE 열에 크기가 표시되면(약 190~280MB — CPU 종류에 따라 다르고, 애플 실리콘 맥은 280MB 쪽입니다) 성공입니다.

3단계 — nginx 컨테이너로 생명주기 한 바퀴

bash

```
1 # 백그라운드(-d)로 실행, 내 8080 포트를 컨테이너 80에 연결
2 docker run -d --name web -p 8080:80 nginx:1.27
3
4 # 실행 중인 컨테이너 확인
5 docker ps
6
7 # 접속 확인 (브라우저에서 http://localhost:8080 도 열어 보기)
8 curl http://localhost:8080
9
10 # 로그 확인
11 docker logs web
12
13 # 정지 → 정지된 것까지 보기 → 삭제
14 docker stop web
15 docker ps
16 docker ps -a
17 docker rm web
18 docker ps -a
```

확인 포인트: curl 결과에 "Welcome to nginx!" HTML이 보여야 합니다.

그리고 docker stop 후 docker ps 에는 안 보이지만 docker ps -a 에는 Exited 상태로 남아 있고, docker rm 후에는 -a 에서도 사라지면 성공입니다.

TIP docker ps 는 "지금 돌고 있는 것"만, docker ps -a 는 "정지된 것까지 전부"를 보여줍니다.

정지된 컨테이너도 이름과 디스크를 차지하고 있어서, 같은 이름으로 다시 run 하면 이름 충돌이 납니다.

오늘 가장 자주 만나실 오류입니다.

1단계의 hello-world 가 Exited 로 남아 있는 것도 보일 텐데 정상입니다 — 6단계에서 한꺼번에 치웁니다.

4단계 — 컨테이너 안으로 들어가 봅시다

bash

```
1 docker run -d --name web2 -p 8080:80 nginx:1.27
2
3 # 컨테이너 내부 셸 접속 (-it: 대화형 터미널)
4 docker exec -it web2 bash
5
6 # ↓ 여기서부터는 컨테이너 안입니다. 1-1에서 배운 명령이 그대로 통합니다.
7 pwd
8 ls /usr/share/nginx/html
9 cat /usr/share/nginx/html/index.html
10 cat /etc/os-release          # 어떤 리눅스인지 확인
11
12 # 나오기
13 exit
```

확인 포인트: 컨테이너 안에서 index.html 내용이 보이고, /etc/os-release 에 Debian 이 찍히면 성공입니다.

내 컴퓨터가 맥이든 윈도우든 컨테이너 안은 리눅스라는 것을 눈으로 확인하는 단계입니다.

5단계 — 컨테이너 안에서 고친 것은 어떻게 되나

방금 들어간 컨테이너의 웹 페이지를 고쳐 보고, 컨테이너를 지웠다 다시 만들면 어떻게 되는지 확인합니다.

이론에서 본 "컨테이너 안의 변경은 사라진다"를 직접 겪는 단계입니다.

bash

```

1 # 컨테이너 안의 웹 페이지를 바꿔 봅니다
2 docker exec web2 bash -c 'echo "<meta charset=utf-8><h1>내가 고친 페이지
  </h1>" > /usr/share/nginx/html/index.html'
3 curl http://localhost:8080
4
5 # 컨테이너를 지우고 같은 이미지로 다시 만듭니다
6 docker stop web2 && docker rm web2
7 docker run -d --name web2 -p 8080:80 nginx:1.27
8 curl http://localhost:8080

```

확인 포인트: 고친 직후에는 "내가 고친 페이지"가 나오지만, 지우고 다시 만든 뒤에는 원래의 "Welcome to nginx!"로 돌아오면 성공입니다.

이미지는 그대로였고 내 변경은 컨테이너에만 있었다는 뜻입니다.

TIP 브라우저에서 글자가 「怨畹畹 ㄹ 췌췌畹」처럼 깨져 보이면 안 된 것이 아닙니다 — 브라우저가 이 HTML 의 인코딩을 EUC-KR 로 잘못 짚은 것입니다. 그래서 명령에 <meta charset=utf-8> 을 함께 넣었습니다. curl 은 바이트를 그대로 보여 주므로 터미널에서는 처음부터 제대로 보입니다. 실제 웹 페이지가 항상 charset 을 선언하는 이유가 이것입니다.

중간에 꼬였으면 — 8080 을 다른 컨테이너가 잡고 있거나, exec 가 「is not running」이라 하거나, 고쳤는데 옛 페이지가 나오면 — **원인을 찾지 말고 되돌려서 다시** 합니다. 20초면 됩니다.

bash

```

1 # ① 둘 다 강제로 치웁니다 (없으면 오류가 나지만 무시해도 됩니다)
2 docker rm -f web web2
3
4 # ② 새로 띄우고 고칩니다
5 docker run -d --name web2 -p 8080:80 nginx:1.27
6 docker exec web2 bash -c 'echo "<meta charset=utf-8><h1>내가 고친 페이지
  </h1>" > /usr/share/nginx/html/index.html'
7
8 # ③ 컨테이너 안 파일부터 확인합니다 — 여기서 바뀌어 있어야 합니다
9 docker exec web2 cat /usr/share/nginx/html/index.html
10 #   <meta charset=utf-8><h1>내가 고친 페이지</h1>
11
12 # ④ 그다음 밖에서
13 curl http://localhost:8080

```

③ 은 바뀌었는데 ④ 만 옛 페이지면 8080 을 **도커가 아닌 다른 프로그램**이 잡고 있는 것입니다.

`lsof -nP -iTCP:8080 -sTCP:LISTEN` 으로 누구인지 보고, 그럴 땐 `-p 8081:80` 으로 내 쪽 포트만 바꿔 `localhost:8081` 로 봅니다.

안 보일 때 **컨테이너 안 파일(③)부터 확인** 하는 것이 오늘 익힐 디버깅 습관입니다.

6단계 — 정리하는 습관

bash

```
1 # 이번 실습 컨테이너 정리
2 docker stop web2
3 docker rm web2
4
5 # 정지된 컨테이너 한꺼번에 정리
6 docker container prune
7
8 # 지금 상태 확인 - 아무것도 안 남아야 합니다
9 docker ps -a
```

주의 docker system prune -a 는 안 쓰는 이미지까지 전부 지웁니다.

편하지만 되돌릴 수 없고, 다음 실습에서 큰 이미지를 다시 받느라 시간이 걸립니다.

지금은 container prune 까지만 쓰세요.

└ 도커 설치하기 — 운영체제별 (설치 안되었을 때)

아직 docker 명령이 없는 사람이 보는 절입니다. 세 운영체제 탭 가운데 자기 것만 위에서부터 따라 하면 됩니다. 이미 되는 사람은 건너뛰니다.

참고 아래 세 줄이 이미 통과하면 이 절은 건너뛰고 다음 절 「설치·버전 재확인」으로 가면 됩니다.

명령은 [환경 설정] 페이지와 같습니다 — 여기서는 수업 중에 바로 찾을 수 있게 모아 두었습니다.

bash

```
1 docker version           # Client 와 Server 가 둘 다 나와야 합니다
2 docker compose version   # 1-5 부터 씁니다
3 docker run --rm hello-world # "Hello from Docker!" 가 찍히면 끝
```

Windows 11

WSL2 위에 Docker Desktop 을 올리는 순서입니다.

다섯 단계, 중간에 재부팅이 한 번 있습니다.

1단계. 시작 메뉴에서 **PowerShell** 을 마우스 오른쪽 버튼으로 눌러 「관리자 권한으로 실행」 합니다.

powershell

```
1 wsl --install -d Ubuntu-22.04
```

2단계. 안내대로 **재부팅** 합니다.

우분투 창이 저절로 뜨면 **사용자 이름과 비밀번호** 를 만듭니다.

비밀번호는 쳐도 화면에 안 보이는 것이 정상입니다.

3단계. WSL 버전을 확인합니다(PowerShell).

powershell

```
1 wsl -l -v
2 #   NAME                STATE      VERSION
3 #   Ubuntu-22.04        Running    2          ← 반드시 2
4
5 # 1 이면 올립니다 (몇 분 걸립니다)
6 wsl --set-version Ubuntu-22.04 2
```

4단계. docker.com 에서 **Docker Desktop** 을 내려받아 설치합니다.

설치 화면에서 **「Use WSL 2 based engine」** 을 체크합니다.

설치가 끝나면 Desktop 을 열고 **Settings → Resources → WSL Integration → Ubuntu-22.04** 를 클릭합니다.

5단계. 시작 메뉴의 **Ubuntu** 로 우분투 터미널을 **새로** 열고, 맨 위의 세 줄로 확인합니다.

막히면 — 화면에 나온 것으로 찾기

화면에 나온 것	어디서 막힌 것	처방
<code>wsl -l -v</code> 에 배포판이 없다 · <code>wsl</code> 명령이 없다	WSL 자체가 없다	1단계부터. 재부팅을 빠뜨리지 않는다
Ubuntu-22.04 ... VERSION 1	WSL1 이라 도커가 못 붙는다	<code>wsl --set-version Ubuntu-22.04 2</code>
「가상화가 꺼져 있다」 · <code>WslRegisterDistribution failed · 0x80370102</code>	BIOS 의 가상화가 꺼져 있다 — 가장 흔한 원인	재부팅 → BIOS → VT-x(Intel) / SVM(AMD) 켜기
Desktop 설치 중 「WSL 2 installation is incomplete」 · 「requires a newer WSL kernel」	WSL 커널이 옛것	관리자 PowerShell → <code>wsl --update</code> → <code>wsl --shutdown</code> → Desktop 다시 실행
Desktop 은 켜지는데 우분투에서 <code>docker: command not found</code>	WSL Integration 이 꺼져 있다	4단계의 Integration 을 켜고 터미널을 새로 연다
<code>docker version</code> 에 Client 만 있고 Server 가 없다	Desktop 이 아직 뜨는 중	작업 표시줄 고래 아이콘이 멈출 때까지 기다린다(첫 실행 1~2분)
Desktop 설치가 관리자 권한 · 회사 정책에 막힌다	Desktop 을 못 쓰는 노트북	아래 「Desktop 없이」 로 간다 — 오늘 실습에 차이가 없다

Desktop 없이 — WSL2 우분투 안에 직접 설치

WSL2 우분투만 있으면 그 안에 도커를 직접 갑니다.

회사 정책에도 걸리지 않고 완전 무료입니다.

명령은 리눅스 탭과 같고, **마지막 한 줄만 다릅니다** — WSL2 는 데몬이 자동으로 켜지지 않아 우분투를 새로 열 때마다 `sudo service docker start` 를 칩니다.

bash

```

1 # 우분투(WSL2) 터미널에서 — 리눅스 탭의 1~5 단계와 같습니다
2 sudo apt-get update
3 sudo apt-get install -y ca-certificates curl
4 sudo install -m 0755 -d /etc/apt/keyrings
5 sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
  /etc/apt/keyrings/docker.asc
6 sudo chmod a+r /etc/apt/keyrings/docker.asc
7 echo "deb [arch=$(dpkg --print-architecture) signed-
  by=/etc/apt/keyrings/docker.asc]
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
  $VERSION_CODENAME) stable" | sudo tee
  /etc/apt/sources.list.d/docker.list
8 sudo apt-get update
9 sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-
  compose-plugin
10 sudo usermod -aG docker $USER
11 newgrp docker
12
13 # WSL2 에서만 필요한 한 줄 — 우분투를 새로 열 때마다
14 sudo service docker start
15 docker version
16 docker run --rm hello-world
  
```

macOS

Docker Desktop 하나면 됩니다.

칩 종류만 맞게 받습니다.

bash

```

1 uname -m      # arm64 → Apple Silicon(M1~M4) 판 · x86_64 → Intel 판
  
```

1단계. docker.com 에서 Docker Desktop 을 내려받습니다 — **Apple Silicon / Intel 구분.**

2단계. dmg 를 열어 Docker 를 응용 프로그램에 끌어 넣고 실행합니다.

메뉴바의 **고래 아이콘이 멈출 때까지** 기다립니다(5~30초).

3단계. 터미널을 **새로** 열고 맨 위의 세 줄로 확인합니다.

화면에 나온 것	처방
Cannot connect to the Docker daemon at unix:///Users/.../.docker/run/docker.sock	Desktop 이 아직 안 뜬 것. 응용 프로그램에서 Docker 를 실행하고 기다린다
docker 명령 자체가 없다	설치가 안 끝났거나 터미널을 새로 안 열었다. 터미널을 닫고 다시 연다
이미지 크기가 강의를(190MB)과 다르다	Apple Silicon 은 arm64 판을 받아 280MB 쯤. 정상이다
「no matching manifest for linux/arm64」	arm64 판이 없는 이미지. <code>--platform linux/amd64</code> 를 붙이면 느리지만 돈다. 이 과정 이미지는 대부분 arm64 지원

리눅스 (Ubuntu)

Docker 공식 저장소로 Engine 과 Compose 플러그인을 설치합니다.

위에서부터 순서대로 — [환경 설정] 카드와 같은 다섯 단계입니다.

bash

```
1 # 1. 패키지 목록 갱신 + 설치에 필요한 도구
2 sudo apt-get update
3 sudo apt-get install -y ca-certificates curl
4
5 # 2. Docker 공식 GPG 서명 키 등록 (패키지 위변조 검증용)
6 sudo install -m 0755 -d /etc/apt/keyrings
7 sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
  /etc/apt/keyrings/docker.asc
8 sudo chmod a+r /etc/apt/keyrings/docker.asc
9
10 # 3. 공식 저장소를 apt 소스 목록에 추가 - 아키텍처와 우분투 코드명은 $( ) 가 자동으로 넣
  습니다
11 echo "deb [arch=$(dpkg --print-architecture) signed-
  by=/etc/apt/keyrings/docker.asc]
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
  $VERSION_CODENAME) stable" | sudo tee
  /etc/apt/sources.list.d/docker.list
12
13 # 4. 엔진·CLI·containerd·Compose 플러그인 설치
14 sudo apt-get update
15 sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-
  compose-plugin
16
17 # 5. 내 계정을 docker 그룹에 넣어 sudo 없이 쓰기 - 완전 적용은 로그아웃 후 다시 로그인
18 sudo usermod -aG docker $USER
19 newgrp docker
20 docker ps # 헤더만 나오고 비어 있으면 정상
```

이미 다른 방법으로 깔았다면

간 방법	들어 있는 것	추가로 할 일
<code>sudo apt install docker.io</code> (우분투 자체 패키지)	엔진만	Compose 가 따로 → <code>sudo apt install -y docker-compose-v2</code>
<code>sudo snap install docker</code>	엔진 + Compose	그룹은 <code>sudo addgroup --system docker</code> → <code>sudo adduser \$USER docker</code> → <code>newgrp docker</code> → <code>sudo snap disable docker && sudo snap enable docker</code> . 바인드 마운트는 홈 아래만(이 과정은 전부 <code>~/data-eng</code>)
<code>podman-docker</code>	도커가 아니라 홍내 내는 다른 프로그램	지우고 위 다섯 단계로

화면에 나온 것	처방
permission denied while trying to connect to the Docker daemon socket	오늘은 <code>sudo docker ...</code> 로 진행. 5단계의 그룹 넣기는 저녁에 하고 재로그인
Cannot connect to the Docker daemon	<code>sudo systemctl enable --now docker</code>
Command 'docker' not found	아직 설치 전 — 위 다섯 단계

확인 포인트: 맨 위의 세 줄이 통과하면 설치가 끝난 것입니다.

다음 절 「설치·버전 재확인」은 설치 방법에 따라 갈리는 것을 정리한 곳이니 한 번 훑어 두면 뒤 주차에서 덜 막힙니다.

└ 도커 설치·버전 재확인 — 운영체제별 (10분)

설치한 방법이 사람마다 달라도 오늘 실습이 똑같이 되는지 세 줄로 확인합니다. 막히는 사람은 여기서 갈래를 찾습니다.

| 누구나 같은 세 줄

설치 방법이 무엇이었던 아래 세 줄이 통과하면 오늘 실습은 그대로 됩니다.

버전 번호는 달라도 됩니다 — 이 과정의 명령은 20.10 이상이면 전부 같고, 2026년 현재 배포판은 29.x 입니다.

bash

```
1 # ① 두 조각이 다 있나 - Client 와 Server 가 모두 나와야 합니다
2 docker version
3
4 # ② Compose 플러그인이 있나 - 모래(1-5) 부터 씁니다. 지금 확인해 두면 그날 안 막힙니다
5 docker compose version
6
7 # ③ 실제로 하나 돌려 보기 - 끝나면 스스로 지워지는 --rm
8 docker run --rm hello-world
```

화면에 나온 것	뜻	할 일
Client 는 나오는데 Server 가 없다 · <code>Cannot connect to the Docker daemon</code>	일할 쪽(데몬)이 꺼져 있다	Windows·macOS 는 Docker Desktop 을 켜고 기다린다 · 리눅스는 <code>sudo systemctl start docker</code>
<code>permission denied while trying to connect to the Docker daemon socket</code>	내 계정이 docker 그룹에 없다 (리눅스)	오늘은 <code>sudo docker ...</code> 로 진행하고, 아래 리눅스 탭대로 그룹에 넣는다
<code>docker: 'compose' is not a docker command</code>	Compose 플러그인이 빠져 있다	아래 자기 운영체제 탭의 Compose 항목대로 설치한다
<code>Command 'docker' not found</code>	아직 설치 전이다	[환경 설정] 페이지의 자기 운영체제 카드대로 설치한다
Server 버전이 20.10 보다 낮다	너무 오래된 판이다	지우고 [환경 설정] 대로 다시 설치한다

운영체제별로 갈리는 것

같은 도커라도 **설치 경로가 다르면 들어 있는 것과 권한 처리가 다릅니다.** 자기 것만 보면 됩니다.

Windows (WSL2)

Docker Desktop 을 깔고 「Use WSL 2 based engine」 을 켜 상태가 기준입니다.

명령은 **우분투(WSL2) 터미널**에서 칩니다.

Compose 는 Desktop 에 들어 있습니다.

bash

```
1 docker version | head -20      # Client 와 Server 가 모두 나와야 합니다
2 docker compose version         # Desktop 이면 바로 나옵니다
3 docker context show             # desktop-linux 또는 default 이면 정상
```

증상	원인 · 처방
우분투 터미널에서 <code>docker</code> 를 못 찾는다	Desktop → Settings → Resources → WSL Integration 에서 Ubuntu-22.04 를 켜다. 켜 뒤 터미널을 새로 연다
Windows PowerShell 에서는 되는데 우분투에서 안 된다	같은 WSL Integration 문제다. 이 과정은 우분투 쪽에서 친다
<code>Cannot connect to the Docker daemon</code>	Desktop 이 아직 뜨는 중이거나 꺼져 있다. 고래 아이콘이 멈출 때까지 기다린다

TIP Docker Desktop 을 쓸 수 없는 회사 노트북이면 WSL2 우분투 안에 리눅스와 같은 방법(아래 리눅스 탭)으로 Docker Engine 을 직접 깔 수 있습니다.
 그때는 `sudo service docker start` 로 데몬을 켭니다 — WSL2 는 부팅 때 자동으로 켜지지 않습니다.

macOS

Docker Desktop 하나면 끝입니다.

Compose 도 들어 있습니다.

애플 실리콘(M1~M4)이면 **arm64 판 이미지**를 받으므로 크기가 인텔·윈도와 다르게 보이는 것이 정상입니다(nginx:1.27 이 190MB 가 아니라 280MB 쯤).

bash

```

1 docker version | head -20      # Server 가 나와야 합니다 (예: 29.6.1)
2 docker compose version
3 uname -m                        # arm64 면 애플 실리콘, x86_64 면 인텔

```

증상	원인 · 처방
Cannot connect to the Docker daemon at unix:///Users/.../.docker/run/docker.sock	Desktop 이 꺼져 있다. 응용 프로그램에서 Docker 를 실행하고 5~30초 기다린다
어떤 이미지가 「no matching manifest for linux/arm64」	arm64 판이 없는 이미지다. <code>--platform linux/amd64</code> 를 붙이면 느리지만 돈다. 이 과정 이미지는 대부분 arm64 를 지원한다
<code>docker</code> 명령 자체가 없다	Desktop 설치가 안 끝났거나 터미널을 새로 안 열었다. 터미널을 닫고 다시 연다

리눅스 (Ubuntu)

우분투는 **설치 경로가 세 가지**라 갑니다.

먼저 내가 어느 것으로 깔았는지 확인합니다.

bash

```

1 which docker                # /usr/bin/docker → apt 로 깔았다 .
   /snap/bin/docker → snap 으로 깔았다
2 dpkg -l docker-ce docker.io 2>/dev/null | grep ^ii    # 어느 패키지인지
3 snap list docker 2>/dev/null    # snap 이면 여기 나온다

```

설치 방법	들어 있는 것	알아 둘 것
Docker 공식 저장소 (docker-ce) — [환경 설정] 카드	엔진 + Compose 플러그인	가장 표준. 페이지의 5단계 그대로
우분투 패키지 (<code>sudo apt install docker.io</code>)	엔진만. Compose 는 따로	<code>sudo apt install -y docker-compose-v2</code> 를 한 줄 더 — 안 하면 1-5 에서 막힙니다
snap (<code>sudo snap install docker</code>)	엔진 + Compose	바인드 마운트(<code>-v /경로:...</code>)는 홈 폴더 아래만 됩니다 — 이 과정은 전부 <code>~/data-eng</code> 라 문제 없음. 그룹 설정이 아래처럼 한 줄 더
<code>podman-docker</code>	도커가 아니라 홍내 내는 다른 프로그램	고르지 마세요. 뒤 주차에서 어긋납니다

설치 방법과 무관하게 `permission denied ... docker.sock` 이 나오면 내 계정이 docker 그룹에 없는 것입니다.

오늘은 `sudo docker ...` 로 진행해도 되고, 아래로 넣어 두면 내일부터 sudo 없이 됩니다.

bash

```

1 # apt(공식 저장소 · docker.io) 로 깔았을 때
2 sudo usermod -aG docker $USER
3 newgrp docker # 지금 터미널에만 적용. 완전히 적용하려면 로그아웃 후 다시 로그인
4 docker ps
5
6 # snap 으로 깔았을 때는 그룹을 먼저 만들고 서비스를 다시 켭니다
7 sudo addgroup --system docker
8 sudo adduser $USER docker
9 newgrp docker
10 sudo snap disable docker && sudo snap enable docker
11 docker ps

```

주의 데스크톱 우분투는 설치 직후 데몬이 자동으로 켜집니다.

Cannot connect 가 나오면 `sudo systemctl status docker` 로 보고, 멈춰 있으면 `sudo systemctl enable --now docker` 로 켭니다.

확인 포인트: 세 줄이 전부 통과하고 `docker run --rm hello-world` 가 "Hello from Docker!"
를 찍으면, 설치 방법이 무엇이었던 오늘부터 1-5 까지의 실습이 그대로 됩니다.

볼륨과 포트로 데이터 다루기 (1H)

PostgreSQL 컨테이너를 볼륨과 함께 띄워, 컨테이너를 지워도 데이터가 남는 것을 직접 확인합니다.

1단계 — PostgreSQL을 볼륨과 함께 띄웁니다

bash

```
1 # 볼륨(pgdata)을 데이터 디렉터리에 연결해 실행
2 docker run -d --name pg \
3   -e POSTGRES_PASSWORD=engineer123 \
4   -p 5432:5432 \
5   -v pgdata:/var/lib/postgresql/data \
6   postgres:16
7
8 # 뜨는 데 몇 초 걸립니다(처음엔 데이터 폴더를 초기화하느라 3초쯤). 로그로 준비 완료를 확인
  하세요.
9 docker logs pg | tail -n 5
10
11 # 준비됐는지 묻는 한 줄 - accepting connections 가 나올 때까지 몇 초 기다렸다 다시
  칩니다
12 docker exec pg pg_isready -U postgres
13
14 # 볼륨 생성 확인
15 docker volume ls
16
17 # 컨테이너 안의 psql로 접속해 버전 확인
18 docker exec -it pg psql -U postgres -c "SELECT version();"
```

확인 포인트: docker logs 마지막에 "database system is ready to accept connections" 가 보이고, docker volume ls 에 pgdata 가 있고, psql 결과에 PostgreSQL 16.x 문자열이 나오면 성공입니다.

-e 는 환경변수를 넘기는 옵션입니다.

postgres 이미지는 POSTGRES_PASSWORD 를 반드시 요구하도록 만들어져 있어서, 이 값을 안 주면 컨테이너가 바로 죽습니다.

2단계 — 데이터를 넣습니다

bash

```
1 docker exec -i pg psql -U postgres << 'EOF'
2 CREATE TABLE sensor (id serial PRIMARY KEY, name text, value numeric);
3 INSERT INTO sensor (name, value) VALUES ('temp', 21.5), ('humidity',
4 48.2);
5 SELECT * FROM sensor;
6 EOF
```

확인 포인트: SELECT 결과로 두 행(temp 21.5 · humidity 48.2)이 출력되면 성공입니다.

여기서는 `-it` 가 아니라 `-i` 만 씁니다.

`-t` 는 「터미널을 붙여라」 인데 `<< EOF` 로 글을 흘려 넣을 때는 터미널이 아니라서, `-t` 를 붙이면 「stdin is not a terminal」 로 실패합니다.

직접 타이핑할 때만 `-it` 입니다.

3단계 — 컨테이너를 지웠다가 다시 만듭니다

여기가 오늘의 핵심 실험입니다.

컨테이너를 완전히 삭제한 뒤 같은 볼륨으로 새 컨테이너를 만들면 데이터가 살아 있어야 합니다.

bash

```
1 # 컨테이너를 완전히 삭제 (볼륨은 건드리지 않습니다)
2 docker stop pg
3 docker rm pg
4 docker ps -a          # pg 가 없어야 합니다
5 docker volume ls      # pgdata 는 남아 있어야 합니다
6
7 # 같은 볼륨으로 새 컨테이너 생성
8 docker run -d --name pg \
9   -e POSTGRES_PASSWORD=engineer123 \
10  -p 5432:5432 \
11  -v pgdata:/var/lib/postgresql/data \
12  postgres:16
13
14 # 다시 뜨는 데 몇 초 걸립니다 - 바로 치면 아직 준비 중이라 실패할 수 있습니다
15 sleep 3
16
17 # 아까 넣은 데이터가 그대로 있는지 확인
18 docker exec -it pg psql -U postgres -c "SELECT * FROM sensor;"
```

확인 포인트: 새로 만든 컨테이너인데도 temp·humidity 두 행이 그대로 나오면 성공입니다. 컨테이너는 갈아 끼웠지만 데이터는 볼륨에 남아 있었던 것입니다.

TIP 이 성질이 앞으로 계속 쓰입니다.

4주차 MinIO도, 6주차 Kafka도, 7주차 Qdrant도 전부 볼륨을 붙여 띄웁니다. 컨테이너를 다시 만들어도 쌓아 둔 데이터가 날아가지 않게 하기 위해서입니다.

4단계 — 반대로, 볼륨 없이 하면 어떻게 되는지

bash

```

1 # 볼륨 없이 띄우기 (포트는 5433 으로 - 5432 는 pg 가 쓰고 있습니다)
2 docker run -d --name pg_novo1 \
3   -e POSTGRES_PASSWORD=engineer123 \
4   -p 5433:5432 \
5   postgres:16
6
7 sleep 5
8 docker exec -it pg_novo1 psql -U postgres -c "CREATE TABLE t (id int);
9   INSERT INTO t VALUES (1);"
10
11 docker exec -it pg_novo1 psql -U postgres -c "SELECT * FROM t;"
12
13 # 지웠다 다시 만들기
14 docker stop pg_novo1 && docker rm pg_novo1
15 docker run -d --name pg_novo1 -e POSTGRES_PASSWORD=engineer123 -p
16   5433:5432 postgres:16
17
18 sleep 5
19 docker exec -it pg_novo1 psql -U postgres -c "SELECT * FROM t;"

```

확인 포인트: 마지막 명령에서 relation "t" does not exist 오류가 나와 정상입니다. 볼륨이 없으면 데이터가 사라진다는 것을 실제로 확인한 것입니다.

5단계 — 정리

bash

```
1 # 실습용 컨테이너 정리
2 docker stop pg_novol && docker rm pg_novol
3
4 # pg 는 정지만 해 둡니다 (5차시 실습에서 5432 포트를 다시 씁니다)
5 docker stop pg
6
7 docker ps -a
8 docker volume ls
```

주의 pgdata 볼륨은 지우지 마세요.

docker volume rm 은 되돌릴 수 없습니다.

그리고 pg 컨테이너는 rm 하지 말고 stop 만 해 두세요 — 5차시에서 다시 씁니다.

docker volume ls 에 긴 해시 이름의 볼륨이 보이면 pg_novol 이 자동으로 만든 익명 볼륨입니다(postgres 이미지는 -v 를 안 줘도 자기 볼륨을 하나 만듭니다).

docker volume prune 으로 치워도 됩니다 — pgdata 는 pg 컨테이너가 붙들고 있어 지워지지 않습니다.

정리·회고 (0.5H)

오늘 쓴 명령을 한 장으로 모으고, 남은 컨테이너를 정돈합니다.

도커 명령 요약

하려는 일	명령	자주 쓰는 옵션
이미지 받기	docker pull 저장소: 태그	태그를 꼭 붙인다
이미지 목록	docker images	—
컨테이너 실행	docker run 이미지	-d 백그라운드 · --name 이름 · -p 포트 · -v 볼륨 · -e 환경변수
실행 중 목록	docker ps	-a 정지된 것까지
로그 보기	docker logs 이름	-f 실시간 따라가기 · --tail 20
안으로 들어가기	docker exec -it 이름 bash	이미지에 bash 가 없으면 sh
정지 / 시작	docker stop · docker start	—
삭제	docker rm 이름	정지 후에 지운다
볼륨 목록	docker volume ls	docker volume rm 은 되돌릴 수 없음
정지된 것 일괄 정리	docker container prune	—

컨테이너 정돈 확인

bash

```

1 # 지금 상태 점검 - 내일 실습을 위해 깨끗하게 둡니다
2 docker ps -a
3 docker volume ls
4 docker images

```

확인 포인트: 실행 중인 컨테이너가 없고(pg 는 Exited 상태로 남아 있어도 정상), pgdata 볼륨이 남아 있으며, nginx·postgres 이미지가 목록에 있으면 정돈이 끝난 것입니다.

오늘의 체크리스트

- 이미지와 컨테이너의 관계를 봉어빵 틀 비유로 설명할 수 있다.
- 가상머신과 컨테이너의 차이를 "커널을 공유하는가"로 설명할 수 있다.
- nginx 컨테이너를 띄우고 curl과 브라우저로 접속을 확인했다.
- docker ps 와 docker ps -a 의 차이를 실험으로 확인했다.
- docker exec로 컨테이너 내부에 들어가 리눅스 명령을 실행해 봤다.
- 컨테이너 안에서 고친 파일이 재생성 후 사라지는 것을 직접 봤다.
- postgres:16 컨테이너를 볼륨과 함께 실행하고 psql 접속에 성공했다.
- 컨테이너를 지웠다 다시 만들어도 볼륨 덕분에 데이터가 남는 것을 확인했다.
- 볼륨 없이 만든 컨테이너에서는 데이터가 사라지는 것도 확인했다.

자주 겪는 오류

주의 증상: Cannot connect to the Docker daemon → 원인: Docker Desktop(데몬)이 실행되고 있지 않습니다.

→ 해결: Docker Desktop을 실행하고 상태 아이콘이 켜질 때까지 기다린 뒤 docker version 으로 Server 정보가 나오는지 확인합니다.

주의 증상: port is already allocated → 원인: 해당 포트(8080, 5432 등)를 다른 컨테이너나 프로그램이 이미 쓰고 있습니다.

→ 해결: docker ps로 점유 중인 컨테이너를 찾아 stop 하거나, -p 8081:80 처럼 내 쪽 포트를 다른 번호로 바꿔 실행합니다.

주의 증상: The container name "/web" is already in use → 원인: 같은 이름의 컨테이너가 (정지 상태로라도) 남아 있습니다.

→ 해결: docker ps -a 로 확인한 뒤 docker rm web 으로 지우고 다시 실행하거나 --name 에 다른 이름을 줍니다.

예상 질문

- Q. 컨테이너가 바로 죽어요 — docker logs 이름으로 로그를 먼저 보세요. 대부분 필수 환경변수가 빠졌거나(예: POSTGRES_PASSWORD) 설정이 틀린 경우입니다.
- Q. docker exec 에서 bash 가 없다고 나와요 — 가벼운 이미지(alpine 계열)에는 bash 가 없습니다. sh 로 바꿔 docker exec -it 이름 sh 를 쓰세요.
- Q. 컨테이너를 지우면 이미지도 지워지나요? — 아닙니다. docker rm 은 컨테이너만, docker rmi 가 이미지를 지웁니다.
- Q. 맥 M1/M2 인데 이미지가 안 받아져요 — arm64 지원이 없는 이미지입니다. --platform linux/amd64 를 붙이면 대개 돌지만 느려집니다. 이번 과정 이미지는 대부분 arm64 를 지원합니다.

심화 과제 (선택)

- 바인드 마운트를 써 보세요. docker run --rm -v ~/data-eng/week1:/work -w /work ubuntu:22.04 ls 로 내 폴더가 컨테이너 안에서 보이는지 확인합니다.
- nginx 컨테이너에 내 HTML 파일을 바인드 마운트로 넣어 브라우저에 띄워 보세요. (힌트: -v ~/내폴더:/usr/share/nginx/html)
- docker stats 로 실행 중인 컨테이너의 메모리·CPU 사용량을 확인해 보세요. 5주차 메모리 관리에서 다시 씁니다.

참고 링크

- Docker 시작 가이드: <https://docs.docker.com/get-started/>
- Docker CLI 레퍼런스: <https://docs.docker.com/reference/cli/docker/>
- postgres 공식 이미지: https://hub.docker.com/_/postgres

이번 주 산출물

제출은 [산출물 제출](#) 페이지에서 하고, 제출 후 [주차별 산출물 갤러리](#)의 해당 주 섹션에 [1주] 이름 — 산출물명 형식으로 올려 동료와 공유해 주세요.